

API Request Handler

API Request handler is an API server that can handle multiple endpoints, returning dummy data based on the requested resource. Specifically, it serves financial data, sales data, and employee statistics for a company. The application also includes caching mechanisms to reduce the load on fetching data by reusing previously fetched results when the same request comes in. Initial data is calculated at once per request and send as response data to all remaining api request of a company.

----> File Structure

```
.gitignore
go.mod
README.md

+---.dist
+---cmd
|   main.go
+---doc
|   API Request Handler.pdf
+---exe
|   api-request-handler.exe
+---internal
|   +---api
|   |   handler.go
|   +---cache
|   |   cache.go
|   \---services
|       services.go
\---test
    cache_test.go
    handler_test.go
    services_test.go
```

----> Components

- **Entry point (main.go):** The main.go file serves as the entry point for the application. It sets up and starts the API server, configuring routes and starting the HTTP server.
- **Handlers (handler.go):** The handlers define how each API route responds. The FinancialsHandler, SalesDataHandler, and EmployeeStatsHandler method fetches the financial data using a function passed to their respective (DummyFinancialData, DummySalesData, DummyEmployeeStats).
- **Cache (cache.go):** The ConcurrentCache struct manages an in-memory cache. The cache is locked by key, ensuring thread-safety when fetching or computing data for a specific company. If a cached entry exists and is still valid (based on TTL), it will be returned; otherwise, the system computes the data, stores it in the cache, and notifies all waiting clients.
- **Services (services.go):** Contains dummy data functions (DummyFinancialData, DummySalesData, DummyEmployeeStats) simulating the kind of data that might be fetched or computed.
- **test (cache_test.go):** Tests the functionality of the ConcurrentCache in handling the caching of computed data and ensuring it expires as expected.
- **test (handler_test.go):** Tests the HTTP handler for **“/api/company/financials”**, **“/api/sales/data”**, and for **“/api/employee/stats”** endpoint. It verifies that the handler successfully processes

requests and returns the correct status code.

- **test (services_test.go):** Tests the dummy service functions (DummyFinancialData, DummySalesData, DummyEmployeeStats) to ensure that they return non-nil results.

----> Installation & Usage

Note: This project is built using **Go 1.23.5**. Go installation is must to run the project as shown below

- Clone the repository: `git clone <https://github.com/imofficialvivek/api-request-handler.git>`
- Change the directory and checkout to develop branch
 - `cd api-request-handler`
 - `git checkout develop`
- Execute below command to run the program
 - `go run main.go`
- Run below command in another terminal
 - `curl "http://localhost:8080/api/company/financials?companyId=123"`
 - `curl "http://localhost:8080/api/sales/data?companyId=123"`
 - `curl "http://localhost:8080/api/employee/stats?companyId=123"`

OR

Note: If you want to run the program without installing Go then just execute the windows executable file called “**api-request-handler.exe**” (Windows OS is required), provided in the exe folder of repo.

- Run below command in another terminal
 - `curl "http://localhost:8080/api/company/financials?companyId=123"`
 - `curl "http://localhost:8080/api/sales/data?companyId=123"`
 - `curl "http://localhost:8080/api/employee/stats?companyId=123"`

----> Output for Company ID - 123

– Display message on server side:

-----Starting API Request Handler-----

Server is running on port 8080..

Calculating initial data for: 123-employee

– Response at User side:

```
{"companyId":"123","api":"financials","data":{"company":"Kelp","location":"Andheri"}}
```

```
{"companyId":"123","api":"sales","data":{"company":"Kelp","location":"Andheri"}}
```

```
{"companyId":"123","api":"employee","data":{"company":"Kelp","location":"Andheri"}}
```

----> Testing

-
- **cache_test.go:**
 - **Test Flow:**
 - Initial Computation: A test key ("**testKey**") is added to the cache by calling the **GetOrCompute** method. The computation function returns "**computedValue**", which is then validated.
 - Cache Expiry: After a delay (to simulate TTL expiration), the cache is expected to recompute the value. The test ensures that the recomputed value ("**newComputedValue**") is stored correctly in the cache.
 - **handler_test.go:**
 - **Test Flow:**
 - A **GET** request is simulated to the all the API endpoints.
 - The test checks that the status code of the response is **http.StatusOK (200)**.
 - **services_test.go:**
 - **Test Flow:**
 - The services are tested with a sample company ID ("**id**").
 - The returned data is checked for being non-nil, ensuring that the functions provide valid data when invoked.

---> Testing and its output

- **Cmd:**
 - `go test -v test\cache_test.go`
- **Output:**

```
=== RUN   TestConcurrentCache
Calculating initial data for: testKey
Calculating initial data for: testKey
--- PASS: TestConcurrentCache (60.00s)
PASS
ok      command-line-arguments  60.132s
```

-
- **Cmd:**
 - `go test -v test\handler_test.go`
 - **Output:**

```
=== RUN   TestHandler
Calculating initial data for: test-financials
--- PASS: TestHandler (0.00s)
PASS
ok      command-line-arguments  0.163s
```

-
- **Cmd:**
 - `go test -v test\services_test.go`
 - **Output:**

```
=== RUN   TestDummyServices
--- PASS: TestDummyServices (0.00s)
```

PASS

ok command-line-arguments 0.131s